

Memoria del Proyecto: ShinyVillage

Asignatura: Proyecto Intermodular DAM
Alumno: Cristina García Quintero
Fecha: Marzo 2026
Repositorio: [URL del repositorio GitHub](#)

aquí el resumen en español y en inglés

1. Descripción del proyecto

1.1 Introducción

ShinyVillage es un videojuego de rol en dos dimensiones (RPG 2D) que fue creado con el motor Unity y cuya programación se realizó en C#. La propuesta principal es poner al jugador en la dirección de una aldea, donde tiene que administrar los recursos, cultivar su granja y examinar el ambiente desde un punto de vista cenital típico del género.

El jugador maneja a un personaje que se mueve sin restricciones por el mapa, recoge objetos del entorno, los guarda en un inventario y realiza acciones con aquellos elementos del terreno que son interactivos, como las celdas agrícolas. El personaje es seguido de manera constante por la cámara. Como el progreso se almacena en una base de datos local (SQLite), el jugador tiene la posibilidad de crear múltiples partidas independientes, reanudarlas cuando quiera o eliminarlas desde el menú.

El ciclo de juego proyectado incorpora procedimientos relacionados con la agricultura (cultivar, regar y recoger productos en tiles), manejo del inventario (recoger, amontonar y soltar artículos) e interacción con el mapa a través de un sistema de tiles clasificados según su condición.

1.2 Contexto del proyecto

Aspecto	Implementación
Ámbito y entorno	El proyecto se desarrolla en un contexto académico, sin un cliente externo, donde el propio equipo actúa como desarrollador y receptor técnico del producto. El entorno de desarrollo se basa en Unity con C# , utilizando SQLite integrado mediante NuGetForUnity para la persistencia de datos, mientras que el control de versiones se gestiona con Git mediante ramas de desarrollo. El proyecto parte desde cero con el objetivo de aprender desarrollo de videojuegos y aplicar una arquitectura sólida. Para ello se plantean como elementos clave un sistema de guardado seguro del estado de la partida, un inventario con gestión visual de espacios, movimiento del personaje con animaciones fluidas, un sistema de losetas interactivas basado en Tilemap y una estructura escalable que permita añadir nuevas mecánicas sin modificar el núcleo del sistema.

Aspecto	Implementación
Solución y justificación	La solución elegida estructura el juego en sistemas independientes conectados mediante patrones de diseño. Se utiliza el patrón Singleton en gestores como GameManager , DatabaseManager y SaveGameManager para garantizar una única instancia global en Unity . La persistencia se gestiona con SQLite , lo que permite organizar los datos de forma relacional y facilitar consultas y ampliaciones del sistema. Además, el patrón Repository separa el acceso a la base de datos de la lógica del juego. Por otro lado, el inventario se implementa con una clase independiente del motor y una interfaz de usuario separada para la visualización. Finalmente, el sistema de tiles utiliza el componente Tilemap , gestionado por un TileManager . Todo ello sigue una arquitectura en capas que separa presentación, lógica y datos para mejorar la mantenibilidad y escalabilidad del proyecto.
Destinatarios	El juego contempla como único tipo de usuario el jugador. Su rol dentro del sistema le permite crear y modificar partidas desde el menú, así como el movimiento de un personaje y sus decisiones sobre la interacción con el mapa. También puede gestionar su inventario decidiendo qué objetos soltar y cuáles soltar. No se contempla ningún rol de administrador ni perfil técnico dentro de la experiencia de juego. Toda la gestión interna (base de datos, repositorios, gestores) es transparente para el jugador y opera de forma automática en segundo plano.

1.3 Objetivo del proyecto

ShinyVillage tiene la finalidad de ofrecer al jugador una experiencia de ocio digital tranquilo, de estilo casual basado en una simple gestión de recursos y un entorno de fantasía.

La aplicación no tiene una vocación comercial en este momento porque está enmarcada en un contexto de aprendizaje y desarrollo en técnicas de programación de videojuegos.

1.4 Marco legal

Al tratarse de un videojuego de entretenimiento en fase de desarrollo académico, sin distribución comercial ni recogida de datos personales identificativos, el marco legal aplicable es reducido.

Protección de datos (RGPD / LOPDGDD)

La aplicación almacena únicamente datos de juego —nombre de personaje, progreso, posición— de forma local en el dispositivo del usuario, sin transmisión a servidores externos. El nombre del personaje es un alias libremente elegido, no vinculado a ninguna identidad real, por lo que en su estado actual no se tratan datos personales en el sentido del RGPD (Reglamento UE 2016/679) ni de la LOPDGDD (LO 3/2018). Si en el futuro se incorporasen cuentas de usuario o cualquier dato identificativo, la aplicación quedaría sujeta a dichas normas.

Propiedad intelectual

El proyecto ShinyVillage se distribuye bajo una **licencia personalizada basada en Apache License 2.0**, a la que se añade una cláusula de restricción de uso comercial. Esto significa que cualquier persona puede

usar, estudiar, modificar y redistribuir el software y sus versiones derivadas, pero **no puede hacerlo con fines comerciales** sin autorización expresa.

Esta licencia no está aprobada por la OSI (*Open Source Initiative*) al incluir dicha restricción, pero es plenamente válida desde el punto de vista legal.

Si en algún momento se desea autorizar un uso comercial concreto a un tercero, bastaría con un acuerdo escrito adicional entre las partes, sin necesidad de cambiar esta licencia.

Clasificación por edades

En caso de distribución pública, el sistema PEGI clasificaría previsiblemente el juego como **PEGI 3**, dado su contenido de fantasía sin elementos violentos ni adultos.

2. Acuerdo del proyecto

2.1 Requisitos funcionales y no funcionales

Requisitos funcionales

Capacidades que el programa debe poder ofrecer al usuario final, en este caso el jugador.

ID	Tipo	Requisito
RF-01	Funcional	El jugador puede crear una nueva partida introduciendo un nombre de personaje y un nombre de slot
RF-02	Funcional	El jugador puede cargar una partida guardada previamente desde el menú principal
RF-03	Funcional	El jugador puede borrar una partida existente, con confirmación previa
RF-04	Funcional	El jugador puede sobrescribir una partida con el estado actual de la sesión en curso
RF-05	Funcional	El personaje se desplaza por el mapa en las cuatro direcciones con animación asociada
RF-06	Funcional	El jugador puede recoger objetos del mundo e incorporarlos al inventario
RF-07	Funcional	El jugador puede consultar su inventario y eliminar objetos, que reaparecen en el mapa
RF-08	Funcional	El jugador puede interactuar con tiles del mapa pulsando la tecla de acción
RF-09	Funcional	El estado de la granja (tiles, cultivos, fases de crecimiento) se guarda y recupera por partida

Requisitos no funcionales

Este tipo de requisitos definen las condiciones y estándares de rendimiento y las restricciones del sistema

ID	Tipo	Requisito
RNF-01	No funcional	Los datos de partida se almacenan localmente en el dispositivo del usuario mediante SQLite
RNF-02	No funcional	El sistema de guardado persiste entre escenas sin pérdida de datos
RNF-03	No funcional	La arquitectura debe permitir añadir nuevos sistemas (cultivos, NPCs) sin modificar el núcleo
RNF-04	No funcional	El tiempo de carga de una partida guardada no debe ser perceptible para el usuario
RNF-05	No funcional	El juego debe ejecutarse en PC con un rendimiento estable bajo Unity

2.2 Planificación de tareas y temporalización

El desarrollo se organiza en fases en las que se incorpora un bloque completo y verificable antes de pasar a la siguiente.

Fase 1 · Fundamentos del proyecto

- └─ Configuración del entorno Unity y estructura de carpetas
- └─ Sistema de movimiento del personaje y seguimiento de cámara
- └─ Implementación del Tilemap e interacciones básicas

Fase 2 · Sistema de inventario

- └─ Clase Inventory y lógica de slots (añadir, apilar, eliminar)
- └─ UI del inventario (Inventory_UI, Slot_UI)
- └─ Mecánica de soltar objetos al mundo

Fase 3 · Persistencia y base de datos

- └─ Integración de SQLite mediante NuGetForUnity
- └─ DatabaseManager: conexión, tablas y operaciones CRUD
- └─ Repositorios: SaveSlotRepository, PlayerRepository
- └─ SaveGameManager como punto de entrada único al guardado

Fase 4 · Menú principal y gestión de partidas (En desarrollo)

- └─ MainMenuManager: nueva partida, carga y borrado
- └─ SaveSlotUI: prefab de fila con botones dinámicos
- └─ Integración completa del flujo menú → juego → guardado

Fase 5 · Sistema de granja (en desarrollo)

- └─ FarmRepository y FarmTileRepository
- └─ Lógica de cultivo: plantar, regar, cosechar
- └─ Persistencia del estado de cada tile por partida

2.3 Metodología a seguir

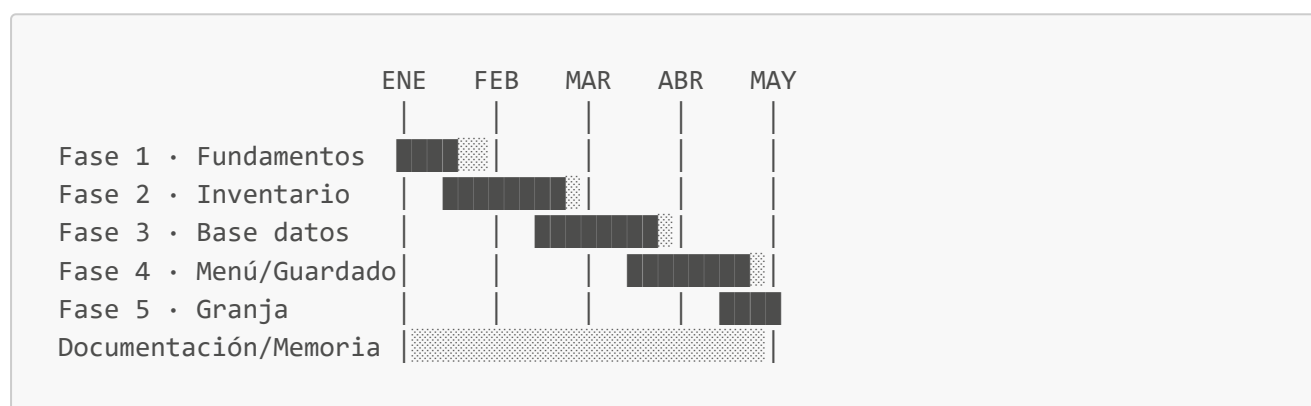
El proyecto adopta una metodología **iterativa e incremental** inspirada en Scrum, que se adapta a un equipo de desarrollo pequeño y a un contexto académico.

Cada fase de la planificación equivale a un sprint con un entregable funcional al final (tarear).

El desarrollo se gestiona mediante Git con ramas independientes por funcionalidad, integrando los cambios a la rama principal una vez verificados. Esta aproximación permite detectar errores de integración de forma temprana y mantener en todo momento una versión jugable del proyecto.

2.4 Temporalización

El diagrama de Gantt siguiente refleja la distribución temporal estimada del proyecto a lo largo del ciclo de desarrollo:



Cada fase tiene una duración aproximada de dos a tres semanas, con solapamiento en la etapa de documentación, que se mantiene activa durante todo el desarrollo.

2.5 Presupuesto

El proyecto no tiene coste económico directo, al desarrollarse en un contexto formativo con herramientas de acceso libre o gratuito para uso académico.

Concepto	Coste
Unity uso personal	0 €
SQLite + NuGetForUnity	0 €
Assets gráficos (libres de uso creados por CupNooble)	0 €
Control de versiones (Git y GitHub)	0 €
Total	0 €

El coste real del proyecto se mide en horas de desarrollo. Estimando una dedicación media de 10 horas semanales durante 5 meses, el esfuerzo total asciende a aproximadamente **200 horas**.

2.6 Contrato y pliego de condiciones/licencia

El proyecto se distribuye con una licencia **Apache 2.0 con una cláusula no comercial** que se describe en el [apartado correspondiente de esta memoria](#).

Como se trata de de un proyecto sin un cliente externo, no hay contrato de prestación de servicios. El acuerdo de desarrollo queda recogido en los terminos de entregas del centro formativo.

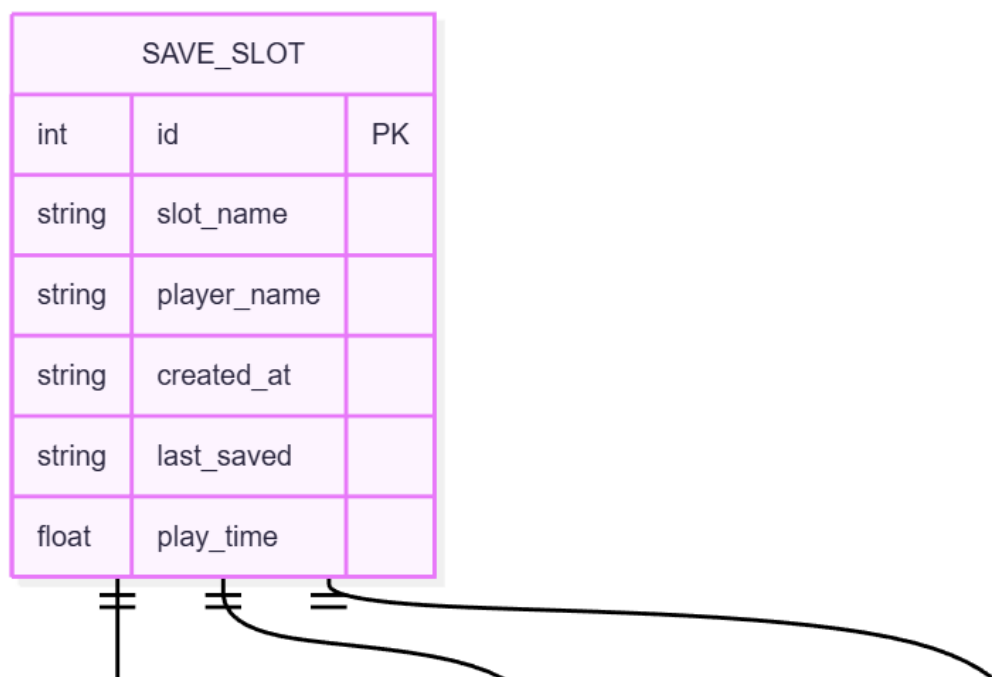
2.7 Análisis de riesgos

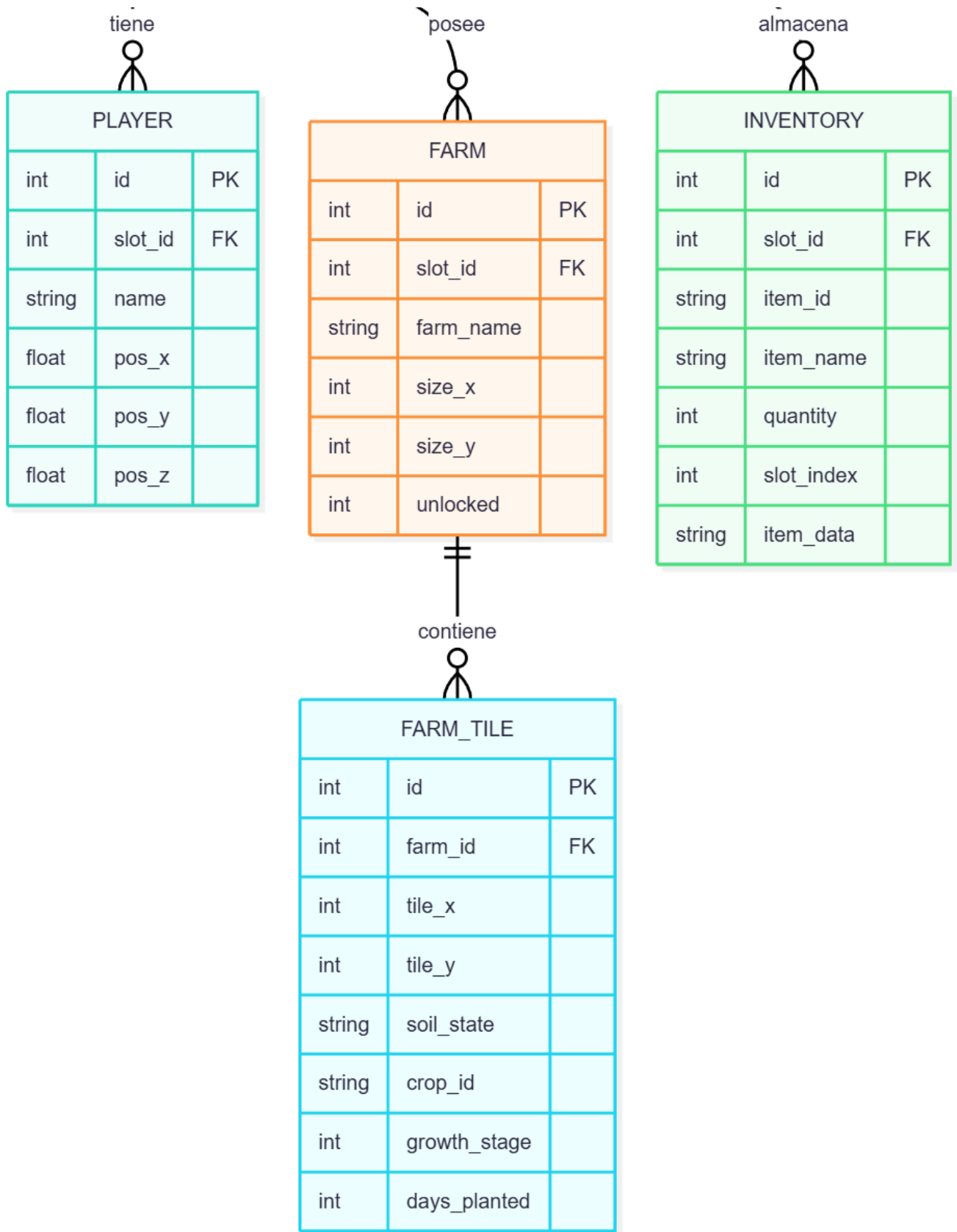
Riesgo	Probabilidad	Impacto	Mitigación
Corrupción de la base de datos SQLite	Baja	Alto	Cierre controlado de conexión en <code>OnApplicationQuit</code> y <code>OnDestroy</code>
Incompatibilidad de la librería SQLite con la versión de Unity	Media	Alto	Uso de NuGetForUnity con versión fijada y probada
Deuda técnica por crecimiento no planificado del código	Media	Medio	Arquitectura en capas con patrón Repository desde el inicio
Pérdida de progreso en el repositorio Git	Baja	Alto	Ramas por funcionalidad y commits frecuentes
Falta de tiempo para completar el sistema de granja	Media	Medio	El núcleo del juego es funcional sin él; se plantea como fase ampliable

3. Análisis y diseño

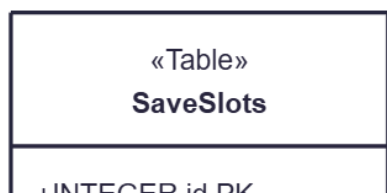
3.1 Modelado de datos

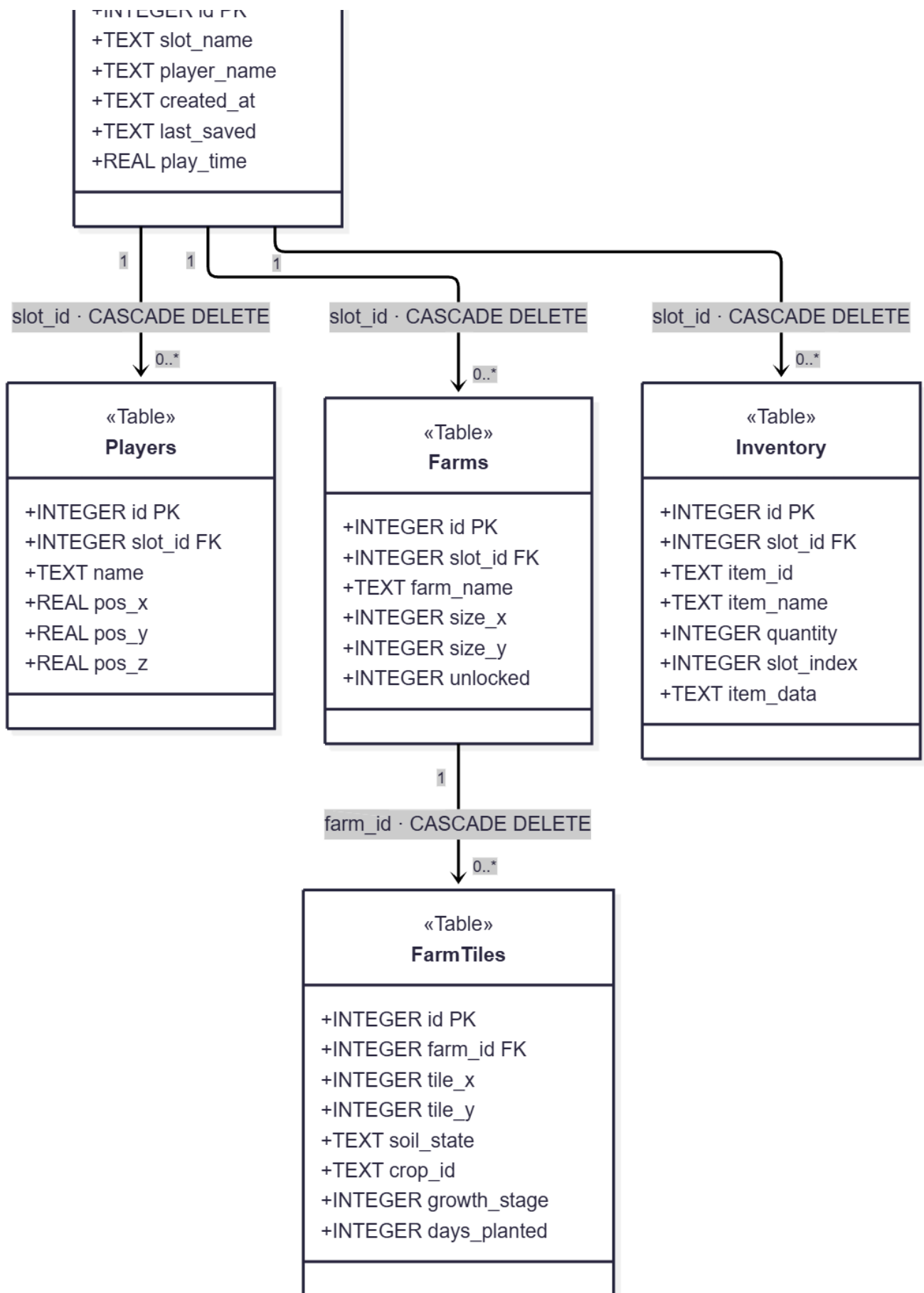
3.1.1 Modelo E/R





3.1.2 Modelo relacional del sistema de guardado





3.1.3 Script de la creación de la BBDD (SQLite)


```

-- =====
-- SCRIPT DE CREACIÓN DE TABLAS – savegame.db
-- =====

-- Tabla raíz: cada fila representa una partida guardada
CREATE TABLE IF NOT EXISTS SaveSlots (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    slot_name     TEXT      NOT NULL,
    player_name   TEXT      NOT NULL,
    created_at    TEXT      NOT NULL,
    last_saved    TEXT      NOT NULL,
    play_time     REAL      DEFAULT 0
);

-- Datos de posición del jugador, ligados a un slot
CREATE TABLE IF NOT EXISTS Players (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    slot_id       INTEGER NOT NULL REFERENCES SaveSlots(id) ON DELETE CASCADE,
    name         TEXT      NOT NULL,
    pos_x        REAL      DEFAULT 0,
    pos_y        REAL      DEFAULT 0,
    pos_z        REAL      DEFAULT 0
);

-- Granjas que pertenecen a un slot
CREATE TABLE IF NOT EXISTS Farms (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    slot_id       INTEGER NOT NULL REFERENCES SaveSlots(id) ON DELETE CASCADE,
    farm_name     TEXT      NOT NULL,
    size_x        INTEGER DEFAULT 10,
    size_y        INTEGER DEFAULT 10,
    unlocked      INTEGER DEFAULT 1    -- 1 = desbloqueada, 0 = bloqueada
);

-- Estado de cada celda/tile dentro de una granja
CREATE TABLE IF NOT EXISTS FarmTiles (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    farm_id       INTEGER NOT NULL REFERENCES Farms(id) ON DELETE CASCADE,
    tile_x        INTEGER NOT NULL,
    tile_y        INTEGER NOT NULL,
    soil_state    TEXT      DEFAULT 'dry', -- dry | watered | fertilized
    crop_id       TEXT      DEFAULT '',
    growth_stage  INTEGER DEFAULT 0,
    days_planted  INTEGER DEFAULT 0
);

-- Inventario del jugador (un ítem por fila)
CREATE TABLE IF NOT EXISTS Inventory (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    slot_id       INTEGER NOT NULL REFERENCES SaveSlots(id) ON DELETE CASCADE,
    item_id       TEXT      NOT NULL,
    item_name     TEXT      NOT NULL,

```

```

quantity    INTEGER DEFAULT 1,
slot_index  INTEGER DEFAULT -1,
item_data    TEXT    DEFAULT '{}' -- JSON para datos extra
);

```

3.2 Análisis y diseño funcional

3.2.1 UML (Clases, secuencia y casos de uso)

Diagramas de clases

Diagrama general

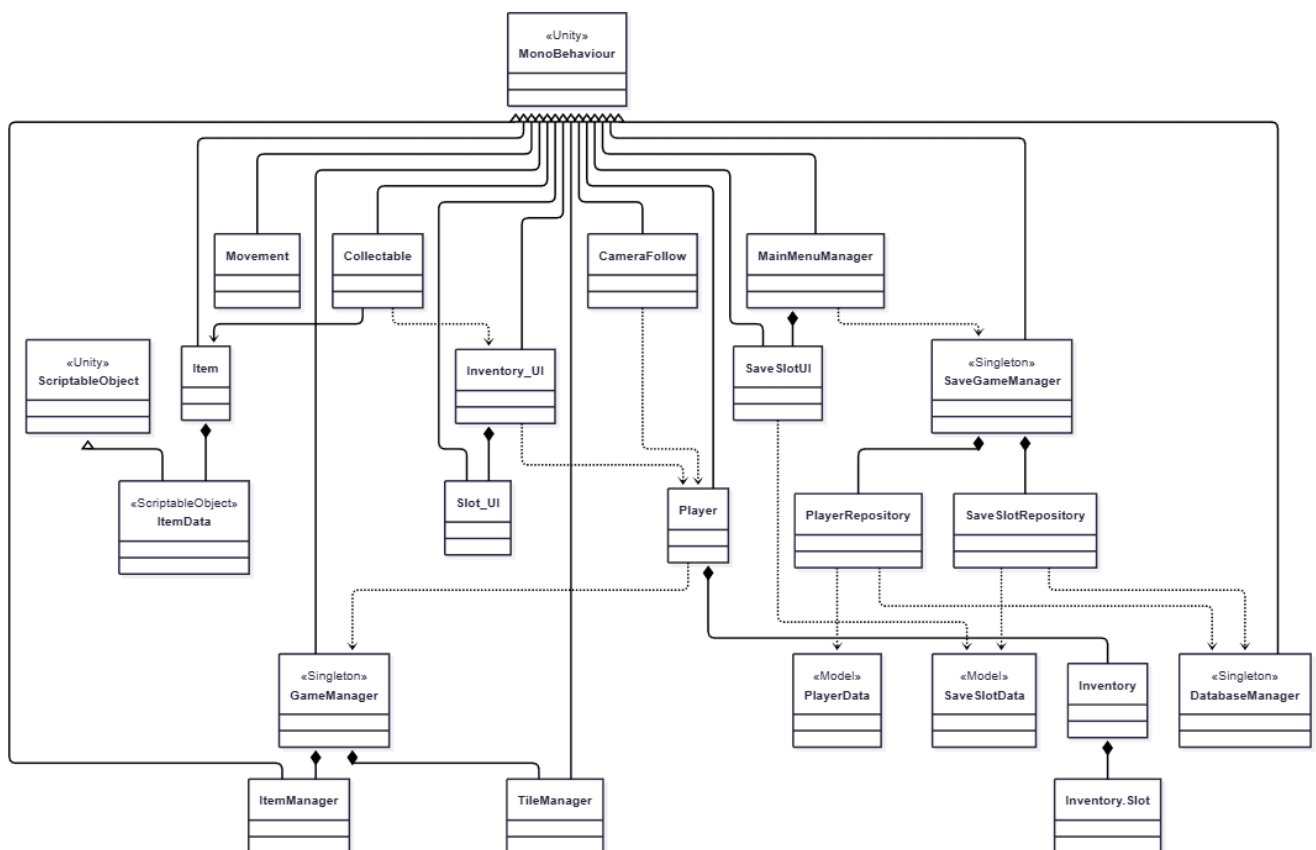


Diagrama desglosado: Gameplay

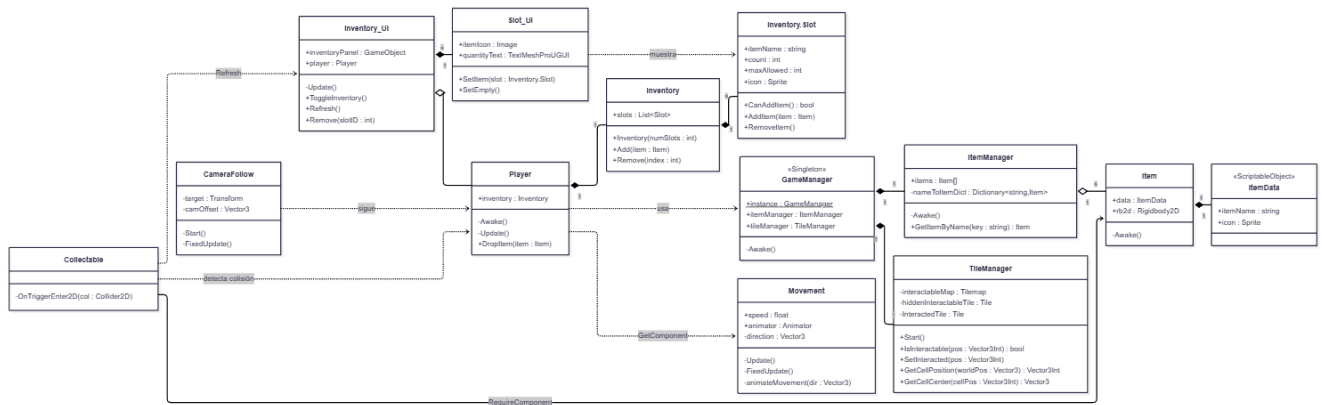
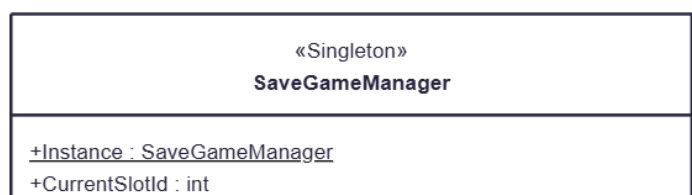


Diagrama desglosado: Saving System (Guardado)



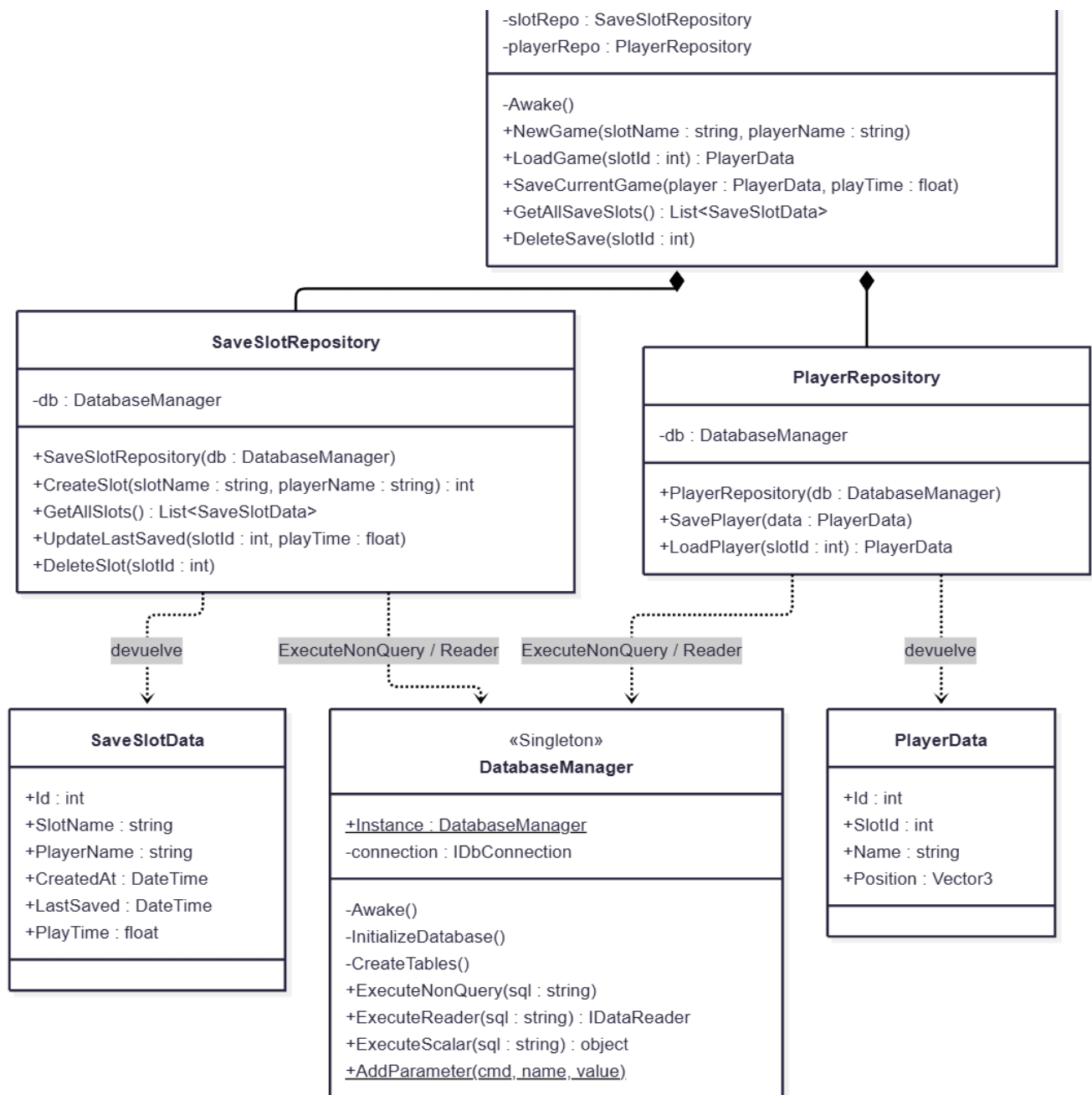
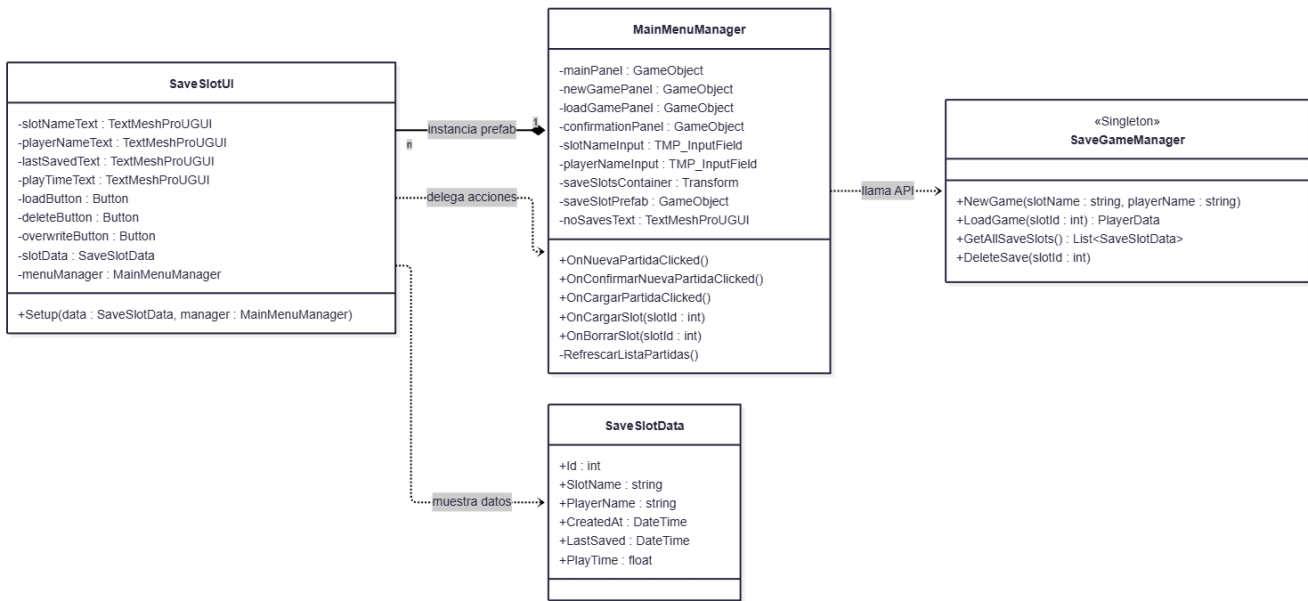
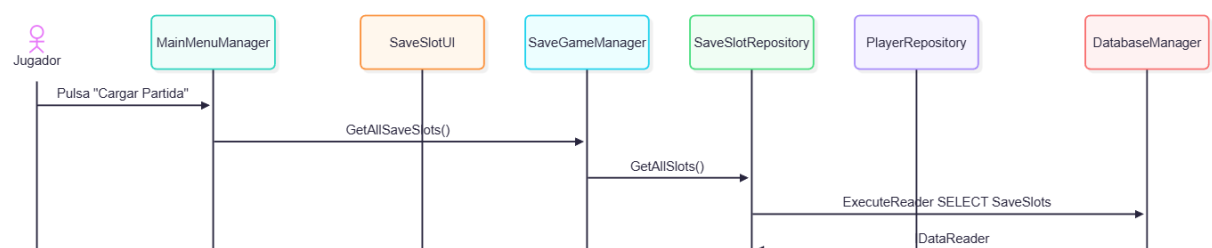
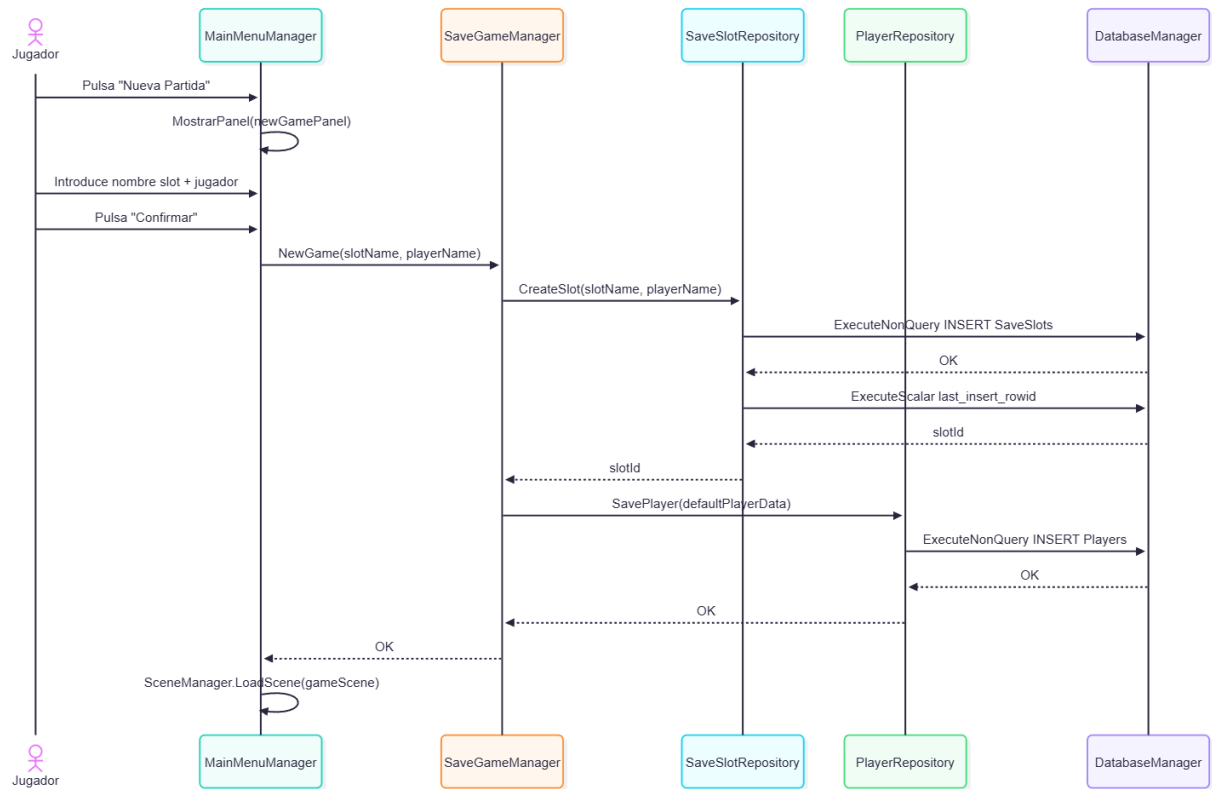


Diagrama desglosado: Main Menu



Diagramas de secuencia



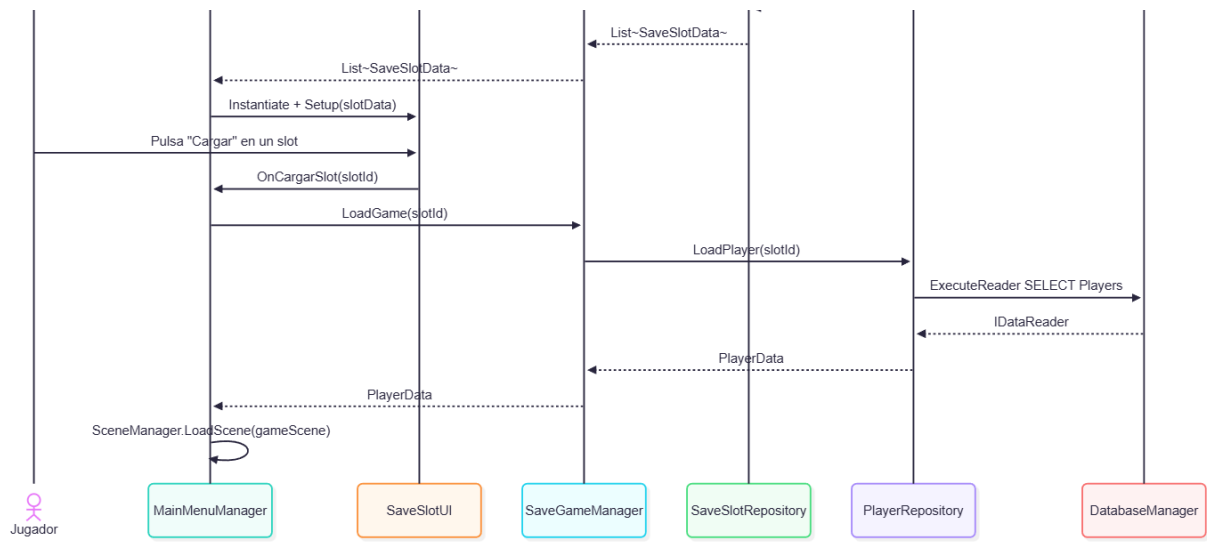
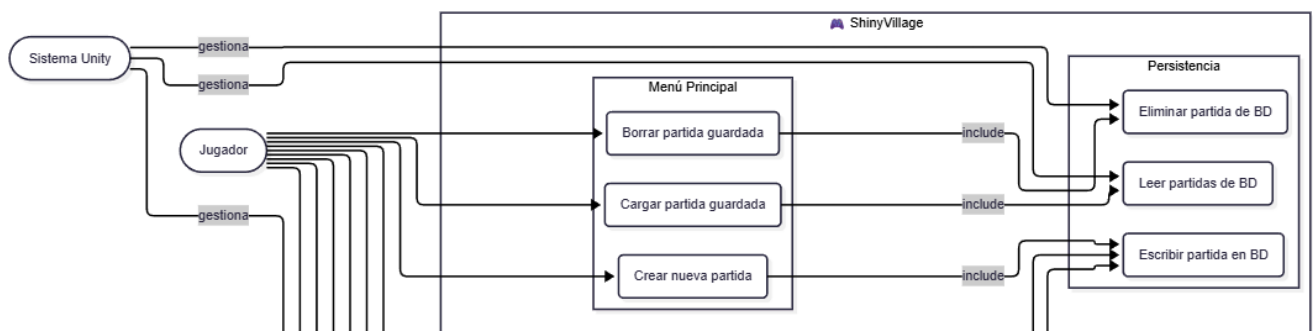
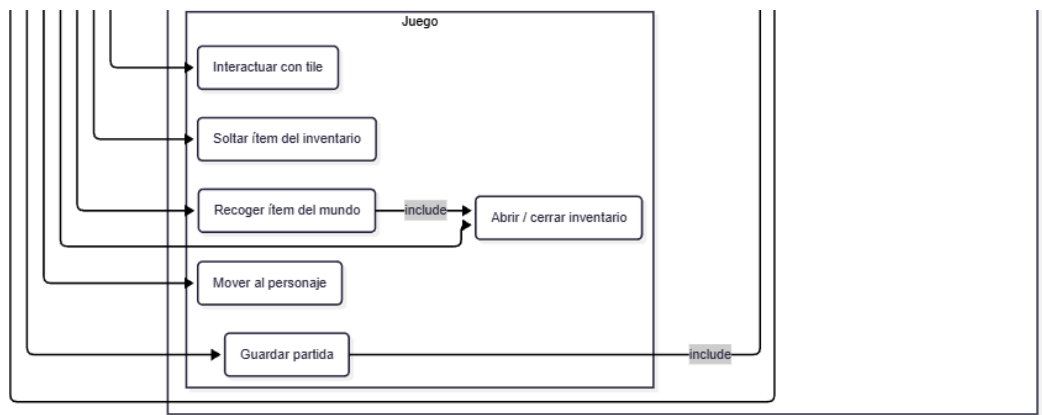
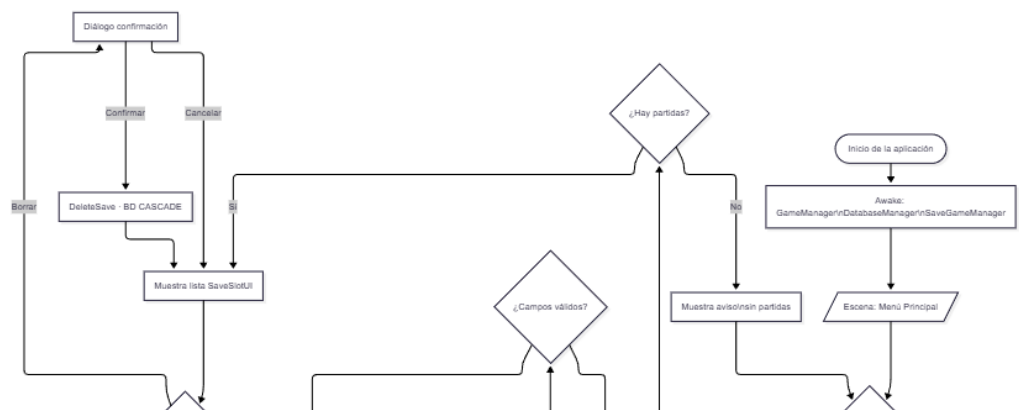


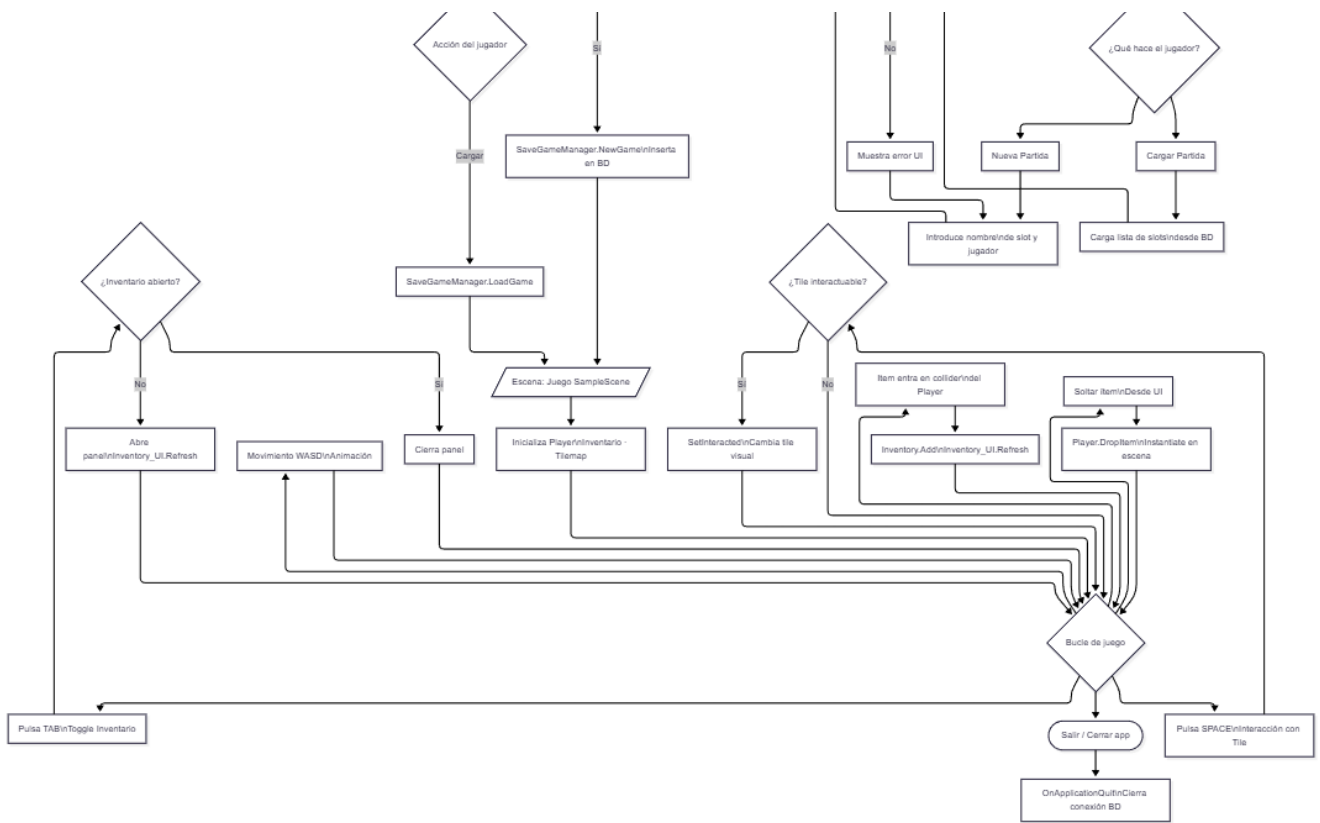
Diagrama de casos de uso



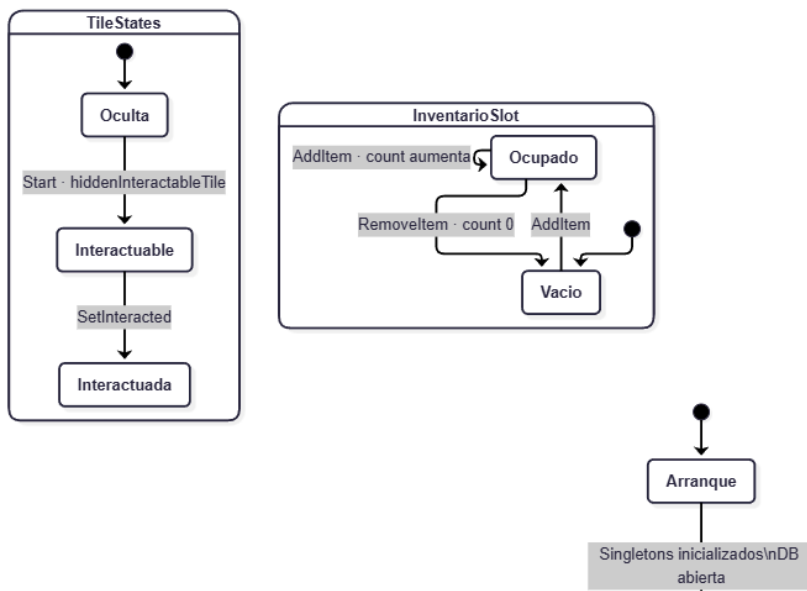


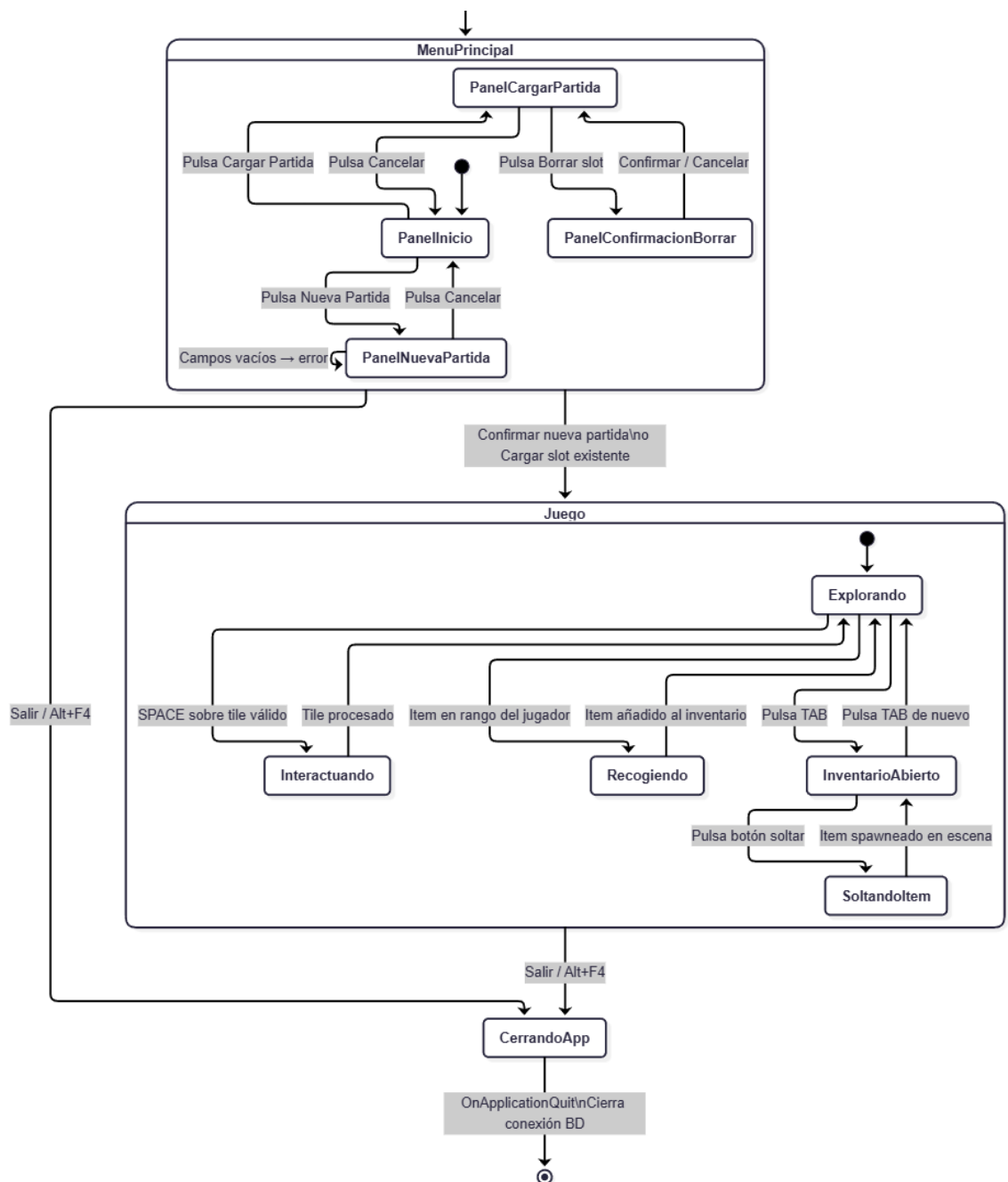
3.2.2 Diagrama de flujo



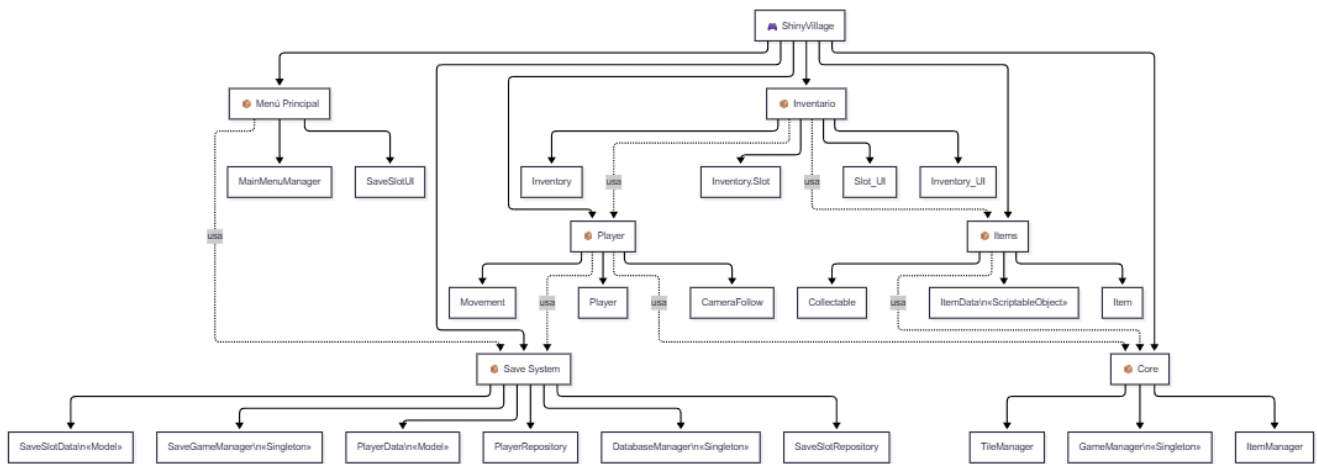


3.2.3 Diagrama de estados





3.2.4 Descomposición en módulos



3.3 Análisis y diseño de la interfaz

3.3.1 Mockups

3.3.2 Diseño visual

3.4 Análisis y diseño de la arquitectura

3.4.1 Tecnologías usadas y herramientas

3.4.2 Arquitectura de los componentes

Explicar la estructura lógica y física del sistema: carpetas, mvc

7. Referencias

Protección de datos y privacidad

- [Reglamento \(UE\) 2016/679 — Reglamento General de Protección de Datos \(RGPD\)](#)
- [Ley Orgánica 3/2018, de Protección de Datos Personales y garantía de los derechos digitales \(LOPDGDD\)](#)
- [Agencia Española de Protección de Datos — Guía para el cumplimiento del RGPD](#)

Propiedad intelectual

- [Real Decreto Legislativo 1/1996 — Ley de Propiedad Intelectual](#)

Clasificación por edades

- [Sistema de clasificación PEGI — Pan European Game Information](#)

Licencias de software

- [Apache License, Version 2.0 — texto oficial](#)
- [Open Source Initiative — definición de licencia de código abierto](#)

Tecnologías utilizadas

- [Unity — documentación oficial](#)
- [SQLite — documentación oficial](#)
- [System.Data.SQLite — documentación del paquete](#)
- [Patrón Singleton en Unity — Game Programming Patterns](#)